

CCE1 EXAMINATION QUESTION BANK — MASTER CONSOLIDATED SOLUTIONS

Course Pattern: TE IT - 2024 Pattern (SPPU)

Academic Term: Semester V (2025-26 / 2026-27)

Document Type: Master Consolidated Solutions (Reundancy Grouped)
Course Code: PEC-304-IT / PEC-321A-IT

IMPORTANT:

Consolidated Question Bank Architecture: This master document synthesizes and groups all redundant and overlapping questions from the official CCE1 Question Bank (covering all 34 questions across Unit 1 and Unit 2) into **15 Comprehensive Topic Solutions**. Every consolidated section explicitly cites all mapped Question IDs, Bloom's Taxonomy Levels (BTL), and marks weightage.

Unit 1: Introduction to Cloud Computing, Services & Deployment Models (CO302.1)

Topic 1.1: Cloud Computing Foundations, Evolution, Characteristics, Advantages & Traditional Comparison

GROUP:

Consolidated Questions Covered:

- **Q1:** Define Cloud Computing. Explain the need, evolution, characteristics, advantages and limitations. [BTL2, 8 Marks]
- **Q5:** Explain characteristics in detail and how they differ from traditional computing. [BTL4, 8 Marks]
- **Q6:** Discuss advantages, limitations, and adoption factors. [BTL4, 8 Marks]
- **Q11:** Explain evolution from traditional to cloud infrastructure. [BTL2, 8 Marks]
- **Q12:** Differentiate traditional vs cloud on scalability, cost, utilization, maintenance, availability. [BTL4, 8 Marks]
- **Q13:** Analyze need, evolution, characteristics, and comparison with traditional approaches. [BTL4, 7 Marks]

1. Statutory NIST Definition of Cloud Computing: According to NIST (National Institute of Standards and Technology):

*"Cloud computing is a model for enabling convenient, **on-demand network access** to a **shared pool of configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released** with **minimal management effort** or service provider interaction."*

2. Need & Economic Drivers:

- **Shift from CapEx to OpEx:** Eliminates upfront investments in hardware, datacenters, cooling, and power, moving to a pay-as-you-go metered operational model.
- **Elastic Scalability:** Dynamic resource allocation matches real-time workload fluctuations.
- **High Availability & DR:** Hyperscale geographic redundancy guarantees business continuity.

- **Agility & Speed to Market:** Environments spin up in seconds via APIs rather than months of hardware procurement.

3. Chronological Evolution Roadmap:

1. **Mainframe Computing (1960s):** Centralized processing via dumb terminals.
2. **Distributed & Client-Server Systems (1980s–90s):** Autonomous networked nodes distributing application tiers.
3. **Cluster & Grid Computing (1990s–2000s):** Heterogeneous geographically dispersed resource pooling for scientific computation.
4. **Utility Computing:** Packaging computing power as a metered utility service.
5. **Virtualization & Cloud-Native Platforms (2006–Present):** Hypervisors abstract hardware into multi-tenant, elastic pools.

4. Five Essential NIST Characteristics:

1. **On-Demand Self-Service:** Automatic provisioning without human intervention from provider staff.
2. **Broad Network Access:** Available over standard protocols via heterogeneous client platforms.
3. **Resource Pooling:** Multi-tenant model dynamically sharing pooled physical/virtual resources.
4. **Rapid Elasticity:** Seamless automatic horizontal and vertical scaling up and down.
5. **Measured Service:** Transparent resource metering (CPU, RAM, storage, bandwidth) enabling pay-per-use billing.

5. Deep Parameter-Based Comparison: Traditional On-Premises vs. Cloud Computing:

Parameter	Traditional On-Premises Computing	Modern Cloud Computing
Scalability	Vertical scaling only (buying bigger hardware); slow procurement taking weeks/months; hard physical limits	Elastic horizontal and vertical scaling in seconds via automated APIs; near-infinite capacity
Cost Model	High upfront Capital Expenditure (CapEx) for hardware, real estate, cooling, plus perpetual software licenses	Zero upfront CapEx; 100% Operational Expenditure (OpEx) with pay-as-you-go utility billing
Resource Utilization	Extremely low (10% to 20%) due to permanent over-provisioning for theoretical peak loads	High average utilization (>80%) enabled by multi-tenant virtualization and dynamic resource pooling
Maintenance & Upgrades	Heavy burden on in-house IT staff for physical server repairs, cable routing, OS patching, and cooling	Maintenance fully abstracted and managed by cloud vendor; automatic software and firmware updates
Availability & DR	High cost to establish secondary backup datacenters; manual disaster recovery failover	Built-in multi-availability zone (AZ) replication, automated failover, and guaranteed 99.99%+ SLAs

6. Advantages, Limitations, and Adoption Decision Factors:

- **Advantages:** Zero maintenance, global latency reduction via edge datacenters, built-in security compliance, continuous automated feature updates.
- **Limitations:** Network latency/outage dependence, vendor lock-in risks, compliance and data residency constraints (GDPR, HIPAA, RBI rules).
- **Adoption Factors:** Total Cost of Ownership (TCO), sensitivity of business data, regulatory governance, and software architecture modernization requirements.

Source: Buyya et al., Ch 1, p. 3-19 | Source: Srinivasan & Suresh, Ch 1, p. 1-28

Topic 1.2: Cloud Service Models (IaaS, PaaS, SaaS) & Startup Application Scenario

GROUP:

Consolidated Questions Covered:

- **Q2:** Explain IaaS, PaaS, and SaaS with examples and compare them. [BTL4, 8 Marks]
- **Q9:** Compare IaaS, PaaS, SaaS with respect to users, management, cost, control, applications. [BTL4, 8 Marks]
- **Q14:** Startup scenario applying IaaS, PaaS, SaaS regarding user responsibilities and deployment. [BTL4, 8 Marks]

1. Architectural Breakdown of Service Models:

- **1. Infrastructure as a Service (IaaS):** Delivers raw compute (VMs), storage, and networking. User manages OS, runtime, middleware, and applications. (Examples: AWS EC2, Azure VMs, Google Compute Engine).
- **2. Platform as a Service (PaaS):** Delivers pre-configured execution runtime and database engines. User manages application source code and data logic only. (Examples: AWS Elastic Beanstalk, Heroku, Google App Engine).
- **3. Software as a Service (SaaS):** Delivers fully managed, ready-to-use software applications over web interfaces. User manages configuration and user access only. (Examples: Google Workspace, Salesforce, Microsoft 365).

2. Parameter-Based Comparison Matrix:

Comparison Parameter	IaaS	PaaS	SaaS
Target Users	System Architects, Cloud Engineers, DevOps Teams	Application Developers, Database Designers, QA Engineers	End Users, Business Executives, General Consumers
Infrastructure Management	Customer manages OS, middleware, patches; Provider manages physical hardware	Customer manages code and data; Provider manages OS, runtime, and hardware	Provider manages 100% of infrastructure, OS, code, patches, and backups
Cost Model	Metered per vCPU hour, RAM GB, Disk IOPS, and data transfer	Metered per application instance runtime, API invocations, and database sizing	Subscription per user seat / month or tiered consumption pricing
Level of Control	Highest control; root/administrative access to virtual operating systems	Moderate control; control over code environment settings and database schemas	Lowest control; user preferences and access rights configuration only
Suitable Applications	Custom legacy migrations, network testing, deep learning pipelines	Rapid web/mobile app development, microservices, API backends	Standard business tools: Email, CRM, ERP, Video Conferencing, Office suites

3. Startup Application Scenario Analysis:

- **SaaS:** Adopted for instant corporate productivity (Google Workspace, Slack, Jira, GitHub) without provisioning servers.
- **PaaS:** Adopted for core customer-facing web and mobile APIs (Node.js/Python on AWS Elastic Beanstalk) for fast CI/CD and automatic load-balancing.
- **IaaS:** Adopted for custom AI model training or specialized low-latency databases (EC2 GPU instances) requiring low-level kernel tuning.

Source: Srinivasan & Suresh, Ch 2, p. 35-55

Topic 1.3: Cloud Deployment Models & Enterprise Multi-Department Recommendation Scenario

GROUP:

Consolidated Questions Covered:

- **Q3:** Explain Public, Private, Hybrid, Community cloud deployment models and applications. [BTL2, 8 Marks]
- **Q10:** Compare Public, Private, Hybrid, Community deployment models with examples. [BTL4, 7 Marks]
- **Q16:** Large organization department scenario: Recommend models for Finance, HR, and Dev teams with justifications. [BTL5, 8 Marks]

1. Deployment Model Comparison Matrix:

Parameter	Public Cloud	Private Cloud	Hybrid Cloud	Community Cloud
Ownership	Third-party Cloud Provider	Single Enterprise / Hosted	Shared (Enterprise + Provider)	Shared by Consortium
Tenancy	Multi-tenant	Single-tenant	Multi + Single Tenant	Multi-organization Tenant
Security Level	Standard / Shared	Maximum / Dedicated	Balanced / Segmented	High / Shared Policies
Scalability	Near-infinite elasticity	Limited to datacenter hardware	Dynamic (Cloud Bursting)	Limited to community pool
Cost Model	Lowest CapEx (100% OpEx)	High CapEx + OpEx	Balanced CapEx/OpEx	Shared Cost Distribution
Representative Examples	AWS, Azure, Google Cloud	OpenStack, VMware Private Cloud	AWS Outposts + Public AWS	GovCloud, BFSI Banking Cloud

2. Enterprise Scenario Recommendations & Justifications:

- **Finance Department → Private Cloud:** Dedicated single-tenant isolation guarantees compliance with statutory banking regulations and protects sensitive payroll and audit records.
- **HR Department → Community Cloud:** Allows secure data exchange, shared recruitment registries, and joint benefits administration among partner organizations.
- **Development & QA Team → Public Cloud:** Delivers instant, elastic self-service environments for rapid CI/CD test automation without occupying expensive on-premises hardware.
- **Enterprise Integration Strategy:** Seamless **Hybrid Cloud** binding on-premises private clusters with public clouds via encrypted VPN/DirectConnect tunnels.

Source: Buyya et al., Ch 1, p. 12-18

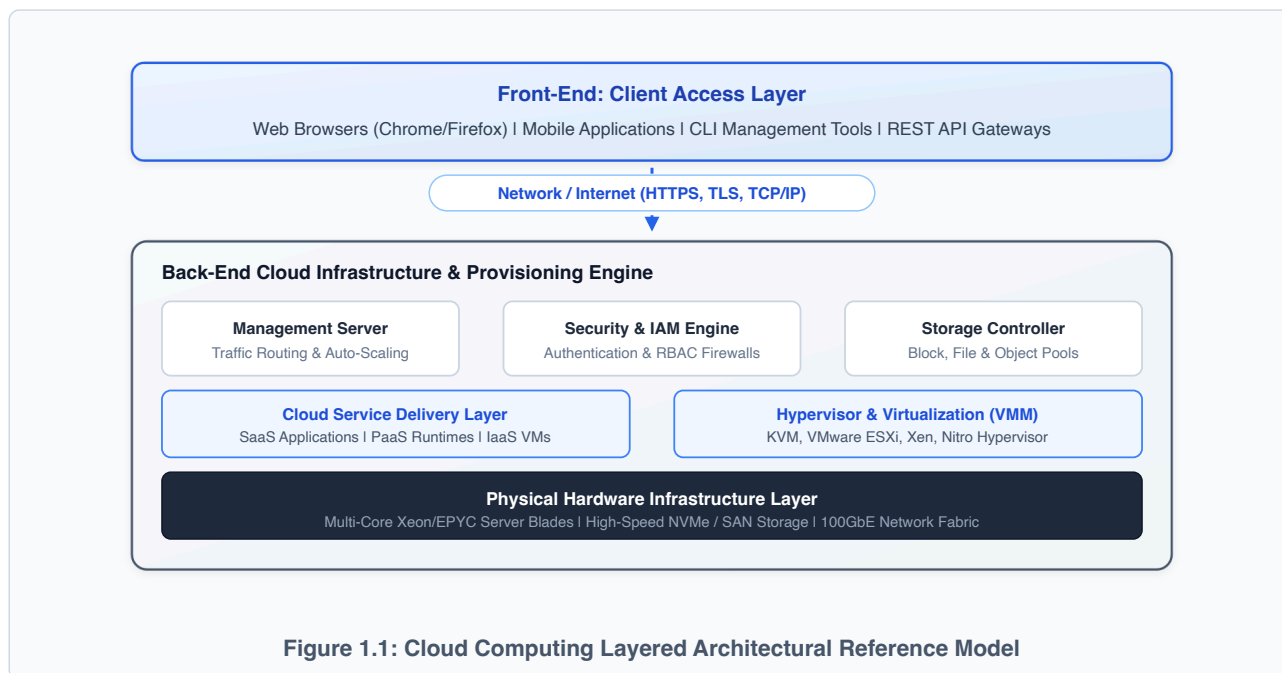
Topic 1.4: Cloud Computing Architecture & Enterprise Evaluation

GROUP:

Consolidated Questions Covered:

- **Q4:** Explain Cloud Computing architecture with a neat diagram and describe major components. [BTL2, 8 Marks]
- **Q15:** Construct and explain architecture; analyze advantages and limitations for enterprise apps. [BTL4, 7 Marks]

1. Layered Reference Architecture & Diagram: Cloud Computing architecture is organized into two primary segments connected across secure network backbones: the **Front-End** (client endpoints) and the **Back-End** (datacenter infrastructure and virtualization platforms).



2. Major Components:

- Client Front-End:** User interfaces, browser endpoints, and client SDKs.
- Management & Orchestration Server:** Directs traffic, allocates compute instances, monitors node health, and executes auto-scaling policies.
- Security & IAM Engine:** Manages authentication, RBAC, and firewall rules.
- Service Delivery Layer:** Hosts runtimes, databases, container clusters, and SaaS endpoints.
- Hypervisor / VMM:** Abstracts host CPUs, RAM, and storage into isolated virtual instances.
- Physical Hardware Layer:** Server racks, distributed storage SANs, and high-speed switches.

Source: Sosinsky, Ch 3, p. 45-65

Topic 1.5: Cloud Storage, Data Management & CAP Theorem

GROUP:

Consolidated Questions Covered:

- Q7:** Explain Cloud Storage and Data Management in detail (importance, benefits, challenges). [BTL2, 8 Marks]

1. Storage Classifications:

- Block Storage (EBS, Managed Disks):** Raw unformatted storage blocks attached to VMs; lowest latency for OS boots and relational databases.
- File Storage (EFS, Azure Files):** Shared hierarchical file system accessible concurrently over NFS/SMB across multiple VMs.
- Object Storage (S3, Blob Storage):** Flat address space storing binary data with unique keys and metadata; infinite scalability for backups and media.

2. Data Management, Consistency & CAP Theorem:

- CAP Theorem:** In distributed cloud storage, a system can provide at most two of three properties: **Consistency (C)**, **Availability (A)**, and **Partition Tolerance (P)**. Across distributed networks, systems trade off between Strong Consistency (CP) and High Availability with Eventual Consistency (AP).

- **ACID vs. BASE:** Relational Cloud SQL enforces ACID properties, whereas NoSQL datastores (DynamoDB, MongoDB) adopt BASE principles for high horizontal scaling.

Source: Buyya et al., Ch 8, p. 215-240

Topic 1.6: Major Cloud Service Providers & Ecosystem Overview

GROUP:

Consolidated Questions Covered:

- **Q8:** Explain the major Cloud service providers and give an overview of their services. [BTL2, 7 Marks]

1. Amazon Web Services (AWS): Global pioneer offering EC2 (compute), S3 (object storage), RDS (relational DB), DynamoDB (NoSQL), and Lambda (serverless). **2. Microsoft Azure:** Enterprise-focused ecosystem featuring Azure VMs, App Services, Blob Storage, Azure SQL, and Azure OpenAI. **3. Google Cloud Platform (GCP):** Industry leader in Big Data and AI/ML, offering Google Compute Engine (GCE), Google Kubernetes Engine (GKE), BigQuery, and Vertex AI. **4. Open-Source Private Cloud Platforms:** OpenStack and Eucalyptus, enabling on-premises AWS-compatible IaaS deployments.

Source: Sosinsky, Ch 4, p. 75-102

Unit 2: Virtualization, Containers & Cloud Infrastructure (CO302.2)

Topic 2.1: Virtualization Foundations, Types, Server Utilization & Infrastructure Applications

GROUP:

Consolidated Questions Covered:

- **Q1:** Define Virtualization, its benefits, applications, and different types with examples. [BTL2, 8 Marks]
- **Q11:** Explain different types of Virtualization and their enterprise applications. [BTL2, 8 Marks]
- **Q12:** Explain how Virtualization improves server utilization, scalability, and resource management. [BTL2, 8 Marks]
- **Q14:** Apply virtualization concepts to cloud infrastructure and differentiate major types. [BTL4, 5 Marks]

1. Definition & Core Concept: Virtualization is a fundamental abstraction technology that decouples physical compute, memory, storage, and network hardware from the operating system and applications, allowing multiple isolated **Virtual Machines (VMs)** to run concurrently on a single physical host server.

2. Taxonomy of Virtualization:

- 1. Server / Compute Virtualization:** Abstracting CPU and RAM via Hypervisors into Virtual Machines (e.g., VMware ESXi, KVM, Xen).
- 2. Storage Virtualization:** Aggregating physical hard drives and SAN arrays into a single virtual pool (e.g., Ceph, VMware vSAN).
- 3. Network Virtualization:** Creating software-defined logical overlay networks decoupled from physical switches (e.g., VMware NSX, VXLANs).
- 4. Desktop Virtualization (VDI):** Hosting desktop environments on datacenter servers and streaming displays to remote thin clients.

3. Server Utilization, Scalability, and Resource Management:

- **Server Consolidation:** Replaces single-application server sprawl (10–15% utilization) with multi-VM density, driving average server utilization past 80%.

- **Dynamic Resource Allocation & DRS:** CPU and RAM can be hot-added to running VMs without rebooting. Dynamic Resource Schedulers automatically live-migrate (vMotion) VMs across hosts to balance cluster loads.
- **High Availability & Fault Tolerance:** Automated failover restarts VMs on surviving healthy nodes if a physical server suffers a hardware fault.

Source: Buyya et al., Ch 3, p. 67-102

Topic 2.2: Hypervisors (Type-I Bare-Metal vs. Type-II Hosted)

GROUP:

Consolidated Questions Covered:

- **Q2:** What is a Hypervisor? Explain Type-I and Type-II Hypervisors with examples. [BTL2, 8 Marks]
- **Q13:** Demonstrate working principles of Type-I and Type-II Hypervisors; analyze architecture, utilization, performance. [BTL4, 5 Marks]

1. Concept of Hypervisor (Virtual Machine Monitor - VMM): A **Hypervisor** is a software/firmware layer that creates and manages Virtual Machines, presenting guest operating systems with virtualized hardware abstractions while intercepting privileged CPU instructions.

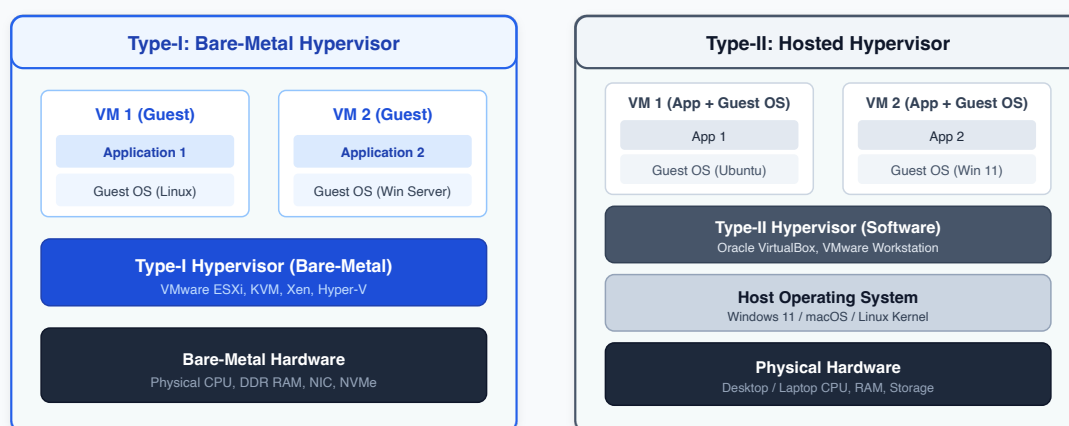


Figure 2.1: Architectural Comparison: Type-I Bare-Metal vs. Type-II Hosted Hypervisor

2. Parameter-Based Comparison Table:

Parameter	Type-I (Bare-Metal Hypervisor)	Type-II (Hosted Hypervisor)
Direct Hardware Access	Direct bare-metal hardware control	Indirect access via Host OS system calls
Resource Overhead	Extremely low (2% to 5% CPU/RAM overhead)	High (Host OS consumes major background resources)
Performance & Throughput	Near-native bare-metal execution speed	Noticeable latency and lower I/O throughput
Primary Use Case	Production enterprise datacenters & cloud clouds	Desktop testing, software development, labs
Representative Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen	Oracle VirtualBox, VMware Workstation

Source: DECAP470, Ch 4, p. 95-112

Topic 2.3: Virtual Machine Architecture & Working Principles

GROUP:

Consolidated Questions Covered:

- **Q3:** Explain Virtual Machine architecture and working with a neat diagram. Discuss advantages. [BTL2, 8 Marks]

1. Architecture & Working: A **Virtual Machine (VM)** packages a full guest operating system, device drivers, and application binaries inside an isolated virtual container managed by the Hypervisor.

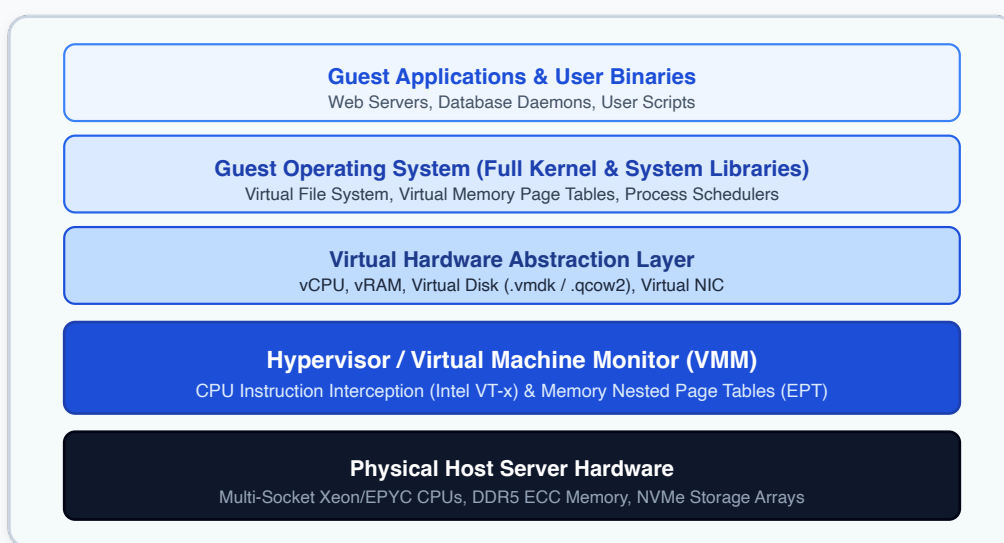


Figure 2.2: Detailed Virtual Machine Architecture

2. Working & Key Advantages:

- **Instruction Execution:** Uses hardware virtualization (Intel VT-x) to run non-privileged guest instructions directly on physical CPU silicon at native speed.
- **Strict Isolation:** Crash or security breach in one VM cannot penetrate other VMs on the physical host.
- **Portability & Snapshots:** Encapsulated as discrete disk images (.vmdk, .qcow2), enabling instantaneous live snapshots and rollback.

Source: Buyya et al., Ch 3, p. 75-92

Topic 2.4: Virtual Machines vs. Containers & Virtualization vs. Containerization

GROUP:

Consolidated Questions Covered:

- **Q4:** Explain Containerization, its benefits, and modern applications. [BTL2, 7 Marks]
- **Q7:** Compare Virtual Machines and Containers on architecture, resources, isolation, performance, startup, portability. [BTL4, 8 Marks]
- **Q10:** Compare Virtualization and Containerization. Explain their importance in Cloud Computing. [BTL4, 8 Marks]
- **Q17:** Analyze and compare Virtual Machines and Containers across architecture, isolation, scalability. [BTL4, 5 Marks]

1. Concept of Containerization: **Containerization** is an OS-level virtualization method where multiple isolated user spaces (containers) share the same underlying host OS kernel. It relies on Linux kernel namespaces (for process/network isolation) and cgroups (for hardware resource metering).

2. Parameter-Based Comparison Matrix:

Parameter	Virtual Machines (VMs)	Containers (Docker / OCI)
Architecture	Hardware-level virtualization; each VM packages its own complete Guest OS, kernel, and virtual drivers	OS-level virtualization; all containers share the host OS kernel and isolate user-spaces via namespaces/cgroups
Resource Utilization	Heavy; each VM consumes gigabytes of dedicated RAM and storage for full OS binaries	Lightweight; containers consume megabytes of RAM, sharing kernel memory and cached image layers
Isolation Level	Extremely strong; hardware-enforced isolation via Hypervisor/CPU rings; impervious to guest kernel panics	Moderate/Process-level; shares host kernel; vulnerability in kernel could theoretically impact host
Performance & Overhead	Slight overhead due to hypervisor CPU trapping and memory page translation layers	Near-native bare-metal execution performance with zero hypervisor instruction overhead
Startup Time	Slow (several seconds to minutes) as it requires a complete operating system boot sequence	Near-instantaneous (milliseconds to seconds); starts as a standard operating system process
Portability & Sizing	Heavy disk images (.iso, .vmdk) ranging from 5GB to 50GB; hypervisor format dependencies	Highly portable lightweight image layers (10MB to 500MB); executes identically on any OCI runtime

Source: Buyya et al., Ch 4, p. 90-105

Topic 2.5: Docker Fundamentals, Architecture & Component Roles

GROUP:

Consolidated Questions Covered:

- **Q5:** Explain Docker fundamentals: Docker Images, Containers, Dockerfile, Docker Hub. [BTL2, 8 Marks]
- **Q16:** Illustrate Docker concepts and explain roles of Images, Containers, Dockerfiles, and Registries. [BTL3, 5 Marks]

1. Core Docker Components & Workflow:

- **1. Dockerfile (Blueprint):** Declarative text script containing build directives (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD).
- **2. Docker Image (Immutable Template):** Read-only layered snapshot packaging code, libraries, and binaries using Union File System (UnionFS).
- **3. Docker Container (Active Instance):** Runnable, isolated execution process instantiated from an image with a thin read-write top layer.
- **4. Docker Registry / Hub (Distribution Central):** Public or private cloud repository for storing, versioning, and sharing container images.

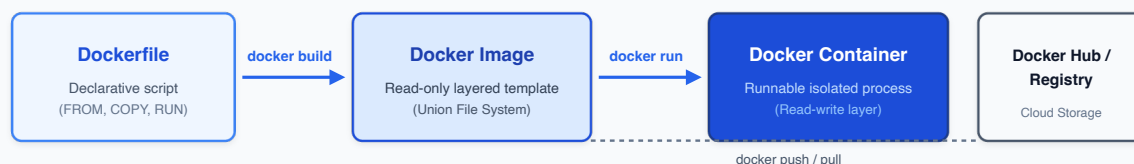


Figure 2.3: Docker Core Workflow Pipeline

Source: Handbook of Cloud Computing, Ch 8, p. 210-226

Topic 2.6: Docker Container Lifecycle & End-to-End Deployment Procedure

GROUP:

Consolidated Questions Covered:

- **Q6:** Explain basic lifecycle of a Docker container with commands/examples. [BTL3, 8 Marks]
- **Q8:** Explain the basic process of Container Deployment using Docker. [BTL3, 8 Marks]
- **Q15:** Demonstrate procedure for deploying application container from image creation to running. [BTL3, 5 Marks]

1. Container Lifecycle States & State Transitions:

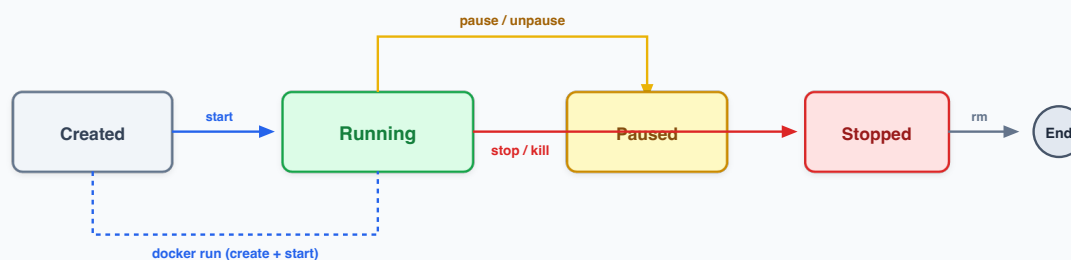


Figure 2.4: Docker Container Lifecycle State Transitions

2. Step-by-Step Deployment Procedure:

1. Write Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

2. **Build Image:** `docker build -t myusername/web-app:1.0 .`
3. **Run Container Locally:** `docker run -d -p 5000:5000 --name web-service myusername/web-app:1.0`
4. **Push to Registry & Deploy:**

```
docker push myusername/web-app:1.0
# On production server:
docker pull myusername/web-app:1.0
docker run -d --restart=always -p 80:5000 --name prod-web myusername/web-app:1.0
```

Source: Handbook of Cloud Computing, Ch 8, p. 220-240

Topic 2.7: Kubernetes Architecture, Need & Container Orchestration Role

GROUP:

Consolidated Questions Covered:

- **Q9:** What is Kubernetes? Explain its need and role in Container Orchestration. [BTL2, 7 Marks]
- **Q18:** Analyze role of Kubernetes in orchestration; explain major components and functions. [BTL4, 5 Marks]

1. Definition & Need for Orchestration: Kubernetes (K8s) automates container scheduling, horizontal scaling, self-healing, and networking across clusters of nodes, solving the operational complexity of managing multi-server microservice deployments.

2. Major Components & Functions:



Figure 2.5: Kubernetes Master-Worker Node Architecture

Source: Buyya et al., Ch 14, p. 380-410